PRACTICAL - 07

Aim: Form validator (SignupForm.java + Driver.java)

Code Screenshot:

Visual Studio Code - SignupForm.java

SignupForm.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class SignupForm {
    @NotBlank
    private String username;

    @MaxLength(12)
    private String city;

    SignupForm(String username, String city) {
        this.username = username;
        this.city = city;
    }
}
```

Visual Studio Code - Driver.java

Driver.java

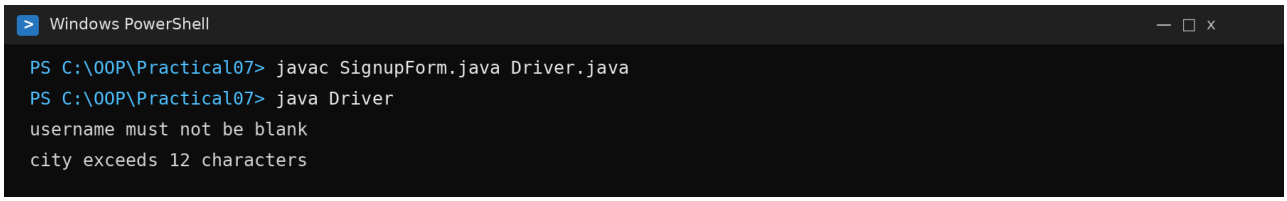
```java
import java.lang.reflect.*;
import java.util.*;

public class Driver {
    static List<String> check(Object obj) {
        List<String> errors = new ArrayList<>();
        for (Field f : obj.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            try {
                Object value = f.get(obj);
                if (f.isAnnotationPresent(NotBlank.class) &&
                    (value == null || value.toString().trim().isEmpty()))
                    errors.add(f.getName() + " must not be blank");

                MaxLength max = f.getAnnotation(MaxLength.class);
                if (max != null && value != null &&
                    value.toString().length() > max.value())
                    errors.add(f.getName() + " exceeds " + max.value() + " characters");
            } catch (IllegalAccessException e) {
                errors.add("Cannot access " + f.getName());
            }
        }
        return errors;
    }

    public static void main(String[] args) {
        SignupForm form = new SignupForm("", "AhmedabadCity");
        for (String error : check(form))
            System.out.println(error);
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to create custom annotations with @Retention and @Target.
- Understood Java Reflection API (Class, Field, getDeclaredFields).
- Learned to inspect and read field annotations dynamically at runtime.
- Implemented a declarative data-validation engine similar to Bean Validation.
- Handled IllegalAccessException through field.setAccessible(true).

PRACTICAL - 07

Aim: Mini test runner (MathTests.java + Driver.java)

Code Screenshot:

Visual Studio Code - MathTests.java

MathTests.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {}

class MathTests {
    @Run
    public void testAddition() {
        System.out.println("Addition: PASS");
    }

    @Run
    public void testSubtraction() {
        System.out.println("Subtraction: PASS");
    }

    public void helper() {
        System.out.println("This method is not a test");
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.lang.reflect.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        MathTests tests = new MathTests();
        int count = 0;

        for (Method m : MathTests.class.getDeclaredMethods()) {
            if (m.isAnnotationPresent(Run.class)) {
                m.invoke(tests);
                count++;
            }
        }
        System.out.println("Tests run: " + count);
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood how testing frameworks like JUnit discover and run test methods.
- Learned method reflection (Method.invoke()) on runtime instances.
- Understood method filtering based on custom annotations.
- Implemented automated discovery and execution pipelines.
- Deepened appreciation for metadata-driven Java programming.

PRACTICAL - 07

Aim: Column mapper (ColumnMapper.java + Driver.java)

Code Screenshot:

Visual Studio Code - ColumnMapper.java

ColumnMapper.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class EmployeeRow {
    @Column(name = "name")
    private String name;

    @Column(name = "email")
    private String email;

    @Column(name = "city")
    private String city = "Not provided";

    void show() {
        System.out.println("Name: " + name);
        System.out.println("Email: " + email);
        System.out.println("City: " + city);
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.lang.reflect.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        String[] header = {"email", "name"};
        String[] row = {"riya@example.com", "Riya"};
        EmployeeRow data = new EmployeeRow();

        for (Field f : EmployeeRow.class.getDeclaredFields()) {
            Column c = f.getAnnotation(Column.class);
            if (c == null) continue;

            for (int i = 0; i < header.length; i++) {
                if (header[i].equals(c.name())) {
                    f.setAccessible(true);
                    f.set(data, row[i]);
                    break;
                }
            }
        }
        data.show();
    }
}
```

Output Screenshot:



Learning Outcome:

- Defined a runtime-retained @Column annotation for field mapping.
- Used reflection to match annotated fields with header names.
- Populated object fields dynamically from a data row.
- Handled missing columns by retaining the field default value.
- Applied annotation-based mapping to a simple data-import task.